



Actividad de Formación Práctica 5

Anti-rebote desarrollado por MEF para garantizar detección positiva de pulsadores mecánicos, aplicación en STM32CubeIDE, programación de microcontroladores.

Integrantes:

Leguizamón, Lautaro Marcelo.

Lucero, Darío Alejandro.

Pistan, Ulises Jesús.

Rivero, Andrés Martín.

Introducción

¿Qué es el rebote mecánico de los botones?

El rebote mecánico ocurre cuando un botón o interruptor mecánico se presiona o suelta. Durante el proceso, los contactos metálicos dentro del botón pueden vibrar y abrirse/cerrarse varias veces antes de asentarse en una posición estable. Esto genera múltiples pulsos eléctricos en lugar de uno solo, lo que puede ser interpretado erróneamente por un microcontrolador como varias pulsaciones. Esto puede causar lecturas erróneas, errores en caso de que realicemos un conteo mediante la activación del botón, y también algunos comportamientos erráticos de nuestra aplicación.

Para mitigar este problema hay soluciones que se implementan mediante hardware:

- Condensador en paralelo
- Circuito Schmitt Trigger
- Uso de Flip-Flop

Pero estas soluciones nos significan un costo adicional y aumentan la complejidad del proyecto, por lo que se opta por una solución mediante software.

Solución al rebote mecánico mediante el uso de MEF

Una de esas soluciones es el uso de una máquina de estados finita con 4 estados:

- **Button Up:** El botón esta sin presionar.
- **Button Falling:** el botón esta en su flanco de bajada, justo en el instante de ser presionado.
- **Button Down:** el botón esta presionado.
- **Button Rising:** el botón esta en su flanco de subida, en el instante en el que se suelta.

Ventajas del uso de MEF para controlar el rebote de botones

- Una máquina de estados garantiza que el botón pase por transiciones claras y controladas entre estados. Esto evita errores al gestionar los cambios de estado solo cuando son estables.
- Permite manejar múltiples botones y eventos complejos con facilidad, ya que cada botón puede tener su propia máquina de estados.
- A diferencia del uso de delays, las máquinas de estados permiten seguir ejecutando otras tareas mientras se espera a que el botón se estabilice. Esto es ideal en sistemas multitarea o de tiempo real.
- Una máquina de estados es fácil de extender para manejar más botones o para integrar más estados y transiciones sin afectar la lógica global del sistema.
- Es posible implementar funcionalidades avanzadas como:
 - a. Pulsaciones largas
 - b. Doble clic
 - c. Combinaciones de botones

Creación de la librería debounce

En primera instancia, creamos la maquina de estados, esta se basa en una enumeración de los estados:

- 1- BUTTON_UP
- 2- BUTTON_FALLING
- 3- BUTTON_DOWN
- 4- BUTTON_RISING

Esta maquina de estados se llamará luego, mediante el tipo definido *debounceState_t*.

Luego declaramos la variable *debounce_timer* que guardara el tiempo entre lecturas que se establece en este caso en 40ms debido al tiempo de respuesta promedio de una persona en presionar dos veces un botón.

Instanciamos la clase mediante *estado_btn* y creamos una variable booleana llamada *lectura_btn* que nos devolverá la lectura final del botón.

debounceFSM_Init

Esta función inicializa la maquina de estados, iniciando en el estado BUTTON_UP, asignando a la variable *debounce_timer* la duración de 40ms, apagando todos los leds, y estableciendo *lectura_btn* en *false*.

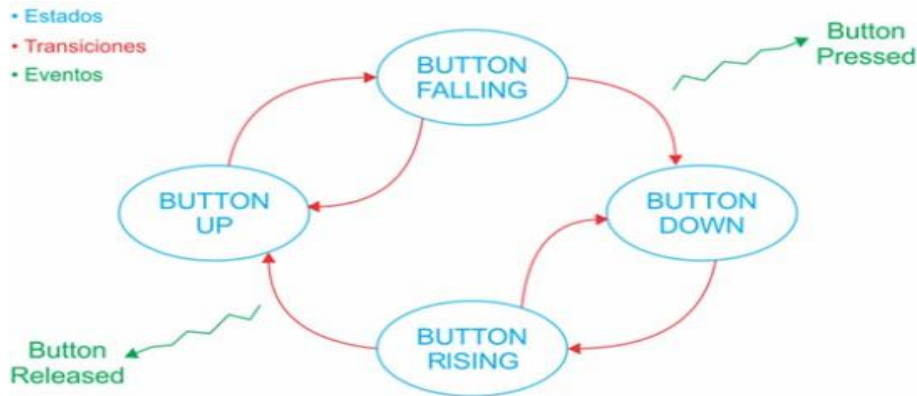
```

1  /*
2  * API_debounce.h
3  *
4  * Created on: 10 Oct. 2025
5  * Author: GRUPO 1_2025:
6  *         LUCERO DARIO
7  *         PISTAN ULISES
8  *         LEGUIZAMON MARCELO
9  *         RIVERO MARTIN
10 */
11
12 #ifndef API_INC_API_DEBOUNCE_H_
13 #define API_INC_API_DEBOUNCE_H_
14
15 #include <stdint.h>
16 #include <stdbool.h>
17
18 /*Includes*/
19 #include "main.h"
20
21 /* Exported types -----*/
22 typedef enum{
23     BUTTON_UP,
24     BUTTON_FALLING,
25     BUTTON_DOWN,
26     BUTTON_RISING
27 } debounceState_t;
28
29 /*Declaracion de prototipo de funciones*/
30 bool readKey(void);
31 void debounceFSM_init(void);
32 void debounceFSM_update(bool buttonRead);
33 void buttonPressed(void);
34 void buttonReleased(void);
35
36 #endif /* API_INC_API_DEBOUNCE_H_ */
37

```

debounceFSM_Update

Esta función controla la maquina de estados en si, recibe el estado mediante *estado_btn*, lee continuamente la señal de entrada del botón mediante *readButton_GPIO()*, y pregunta si ha transcurrido el periodo de 40ms, con esos datos decide cual es el estado siguiente de acuerdo a la siguiente representación:



```

62  * @brief Desarrollo de la MEF debounce
63  * @param boolean buttonRead (tecla presionada)
64  * @retval None
65  */
66 void debounceFSM_update(bool buttonRead)
67 {
68     switch (actualState)
69     {
70     case BUTTON_UP: //estado inicial 0
71         //chequear condicion de transición
72         if(buttonRead == true) //se presionó USER_Btn_Pin
73         {
74             actualState = BUTTON_FALLING; //pasa al estado siguiente
75             delayRead(&debounceDelay); //arranca cuenta de DEBOUNCE_DELAY
76         }
77         break;
78
79     case BUTTON_FALLING:
80         //chequea si paso el tiempo de 40 ms
81         if(delayRead(&debounceDelay))
82         {
83             //chequear condicion de transición
84             if(buttonRead == true) //se presionó USER_Btn_Pin
85             {
86                 buttonPressed(); //dispara funcion buttonPresed()
87                 keyPressed = true; //indica tecla presionada luego de 2 lecturas en 40 ms
88                 fallingEdge = true; //asume que en este estado hubo un flanco decreciente
89                 actualState = BUTTON_DOWN; //pasa al estado siguiente
90             }
91             else
92             {
93                 actualState = BUTTON_UP; // regresa al estado anterior
94             }
95         }
96         break;
97
98     case BUTTON_DOWN:
99         //chequear condicion de transición
100        if(buttonRead == false) //se liberó USER_Btn_Pin
101        {
102            actualState = BUTTON_RISING; //pasa al estado siguiente
103            delayRead(&debounceDelay); //arranca cuenta de DEBOUNCE_DELAY
104        }
105        break;
106
107     case BUTTON_RISING:
108         //chequea si paso el tiempo de 40 ms
109         if(delayRead(&debounceDelay))
110         {
111             //chequear condicion de transición
112             if(buttonRead == false) //se USER_Btn_Pin regresó a estado inactivo
113             {
114                 buttonReleased(); //dispara funcion buttonReleased()
115                 keyPressed = false; //indica tecla presionada luego de 2 lecturas en 40 ms
116                 actualState = BUTTON_UP; //pasa al estado siguiente, el inicial
117             }
118             else
119             {
120                 actualState = BUTTON_DOWN; // regresa al estado anterior
121             }
122         }
123         break;
124
125     default:
126         Error_Handler();
127         break;
128     }
129 }

```

readKey

Esta función nos devuelve la lectura final del botón mediante una variable booleana que se activa exactamente en la transición entre BUTTON_FALLING y BUTTON_DOWN, y que se desactiva cada vez que se llama (para evitar dobles lecturas).

```
30  * @brief Devuelve true si la tecla fue presionada. Asume tecla activa alta como caso normal
31  * F413ZH que estamos usando
32  * @param None
33  * @retval Boolean
34  */
35  bool_t readKey(void)
36  {
37      bool_t keyPress = false;
38      fallingEdge = false;
39
40      if(keyPressed)
41      {
42          keyPress = true;
43      }
44      return keyPress;
45  }
46  }
```

Enlaces

Repositorio grupal

https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025.git

AFP 4

https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025/tree/263d6e811309c4dd68bd65d8ccd1939239c01f70/AFP_5_TDII_2025

App 5.1

https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025/tree/263d6e811309c4dd68bd65d8ccd1939239c01f70/AFP_5_TDII_2025/App_5_1_Grupo_1_2025

App 5.2

https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025/tree/263d6e811309c4dd68bd65d8ccd1939239c01f70/AFP_5_TDII_2025/App_5_2_Grupo_1_2025

App 5.3

https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025/tree/263d6e811309c4dd68bd65d8ccd1939239c01f70/AFP_5_TDII_2025/App_5_3_Grupo_1_2025

App 5.4

https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025/tree/263d6e811309c4dd68bd65d8ccd1939239c01f70/AFP_5_TDII_2025/App_5_4_Grupo_1_2025